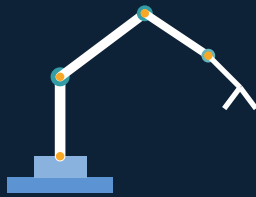

`github.com/nour652/5-DOF-Robot-Arm-Planner-Sorting-System`



5-DOF Robot Arm Planner & Autonomous Sorting System

Technical Documentation & Engineering Report

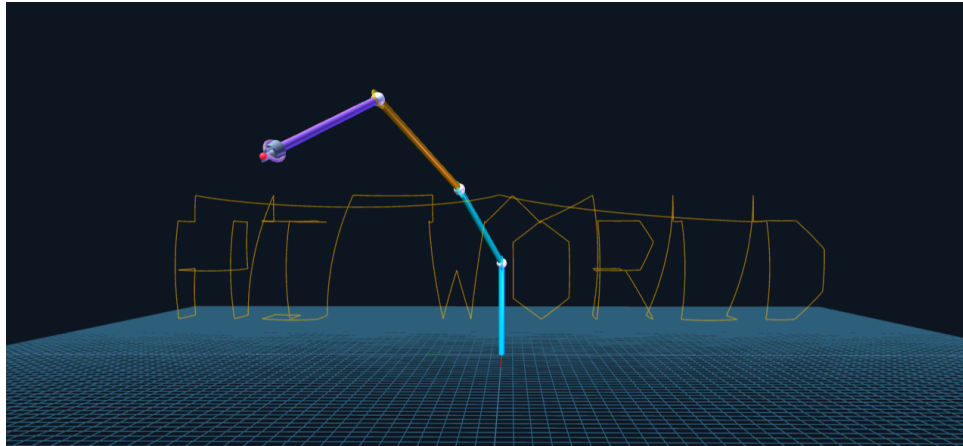
Full-Stack Embedded Robotics with Browser-Based
Kinematic Simulation and ESP32 Firmware

Repository: [nour652/5-DOF-Robot-Arm-Planner-Sorting-System](#)
Platform: ESP32 + PCA9685 + HC-SR04 + LJ12A3
Languages: HTML/JS (77.5%), C++ (18.4%), Python (4.1%)
License: MIT License
Version: 1.0 — Main Branch

Mechatronics & Embedded Systems • Robotics & Automation • IoT / Wi-Fi Communication

A Small Salute to the Reader

Before the technical details begin, the robot says: “Hi World.”



A first gesture from the 5-DOF arm: simulation, path planning, and motion brought together.

Abstract

This report presents the complete technical documentation of a full-stack robotics project that integrates a browser-based kinematic simulation environment with real embedded firmware for the autonomous sorting of objects using a 5-DOF (Five Degrees of Freedom) robotic arm.

The system is architecturally divided into three principal components. The first is a zero-dependency, single-file web application (`robot_arm_planner_sorting.html`) that provides six interactive simulation tabs: Denavit-Hartenberg (DH) parameter editing, Forward Kinematics (FK) manual control, numerical Inverse Kinematics (IK) with a damped-least-squares Jacobian solver, multi-waypoint path planning with preset shape generation, sensor-based sorting path configuration, and microcontroller (MCU) hardware synchronisation.

The second component is an ESP32-based firmware (`robot_arm_esp32_sorting.ino`) that drives six servo motors through a PCA9685 16-channel PWM driver over I²C, implements a 20-endpoint HTTP REST API with CORS support, executes smoothstep-interpolated joint motion at the pulse level, and autonomously performs sensor-gated object sorting using an HC-SR04 ultrasonic proximity sensor and an LJ12A3-4-Z/BX NPN inductive proximity sensor.

The third component is a Python utility (`word_to_path.py`) that converts arbitrary text strings into robot arm handwriting paths by encoding a stroke-based 26-letter font as Cartesian waypoint sequences, which are then uploaded to the planner as JSON path objects and executed via the IK solver.

Together these components form a cohesive, low-cost, Wi-Fi-connected robotic platform capable of simulation, live remote control, path-program upload-and-execute, and sensor-driven autonomous sorting—all without requiring a dedicated ROS or proprietary middleware stack.

Keywords: 5-DOF robot arm, inverse kinematics, Denavit-Hartenberg parameters, ESP32, PCA9685, smoothstep interpolation, sorting automation, embedded web server, Three.js, React, inductive sensor, ultrasonic sensor.

Contents

Abstract	ii
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Document Organisation	2
2 System Overview	3
2.1 High-Level Architecture	3
2.2 Feature Summary	3
2.3 Repository File Structure	5
3 Mechanical Design	6
3.1 5-DOF Arm Configuration	6
3.2 Denavit-Hartenberg Parameters	7
3.3 Workspace Analysis	7
4 Software Architecture	9
4.1 Architectural Philosophy	9
4.2 Browser Planner Architecture	9
4.3 ESP32 Firmware Architecture	9
5 Kinematics	11
5.1 Forward Kinematics	11
5.2 Inverse Kinematics	11
5.2.1 Jacobian Computation	11
5.2.2 Damped Least Squares Update	12
5.2.3 Update and Clamping	12
5.2.4 Convergence Criteria	12
5.2.5 IK Algorithm Summary	13
6 Path Planning	14

6.1	Waypoint-Based Path Representation	14
6.2	Preset Shape Generation	14
6.3	Trajectory Execution	15
7	Sorting System	16
7.1	Sensor Fusion Logic	16
7.2	Rising-Edge Detection	16
7.3	Sorting Decision Flowchart	17
7.4	Sensor Path Configuration	17
8	Control System	18
8.1	Servo-to-PWM Mapping	18
8.1.1	Kinematic-to-Physical Angle Conversion	18
8.2	Smoothstep Interpolation	18
8.3	PCA9685 Driver and I ² C	19
8.4	REST API Reference	19
9	Simulation and Visualisation	21
9.1	3D Viewport (Three.js)	21
9.2	Simulation Tab Screenshots	22
9.3	Embedded ESP32 Web Interface	24
10	Deep Code Analysis	25
10.1	ESP32 Firmware: Key Functions	25
10.1.1	angleToPulse() — Kinematic-to-PWM Conversion	25
10.1.2	moveAll() — Synchronised Smoothstep Motion	25
10.1.3	handleSensorAutomation() — Main Sorting Loop	26
10.2	Python Utility: word_to_path.py	26
11	Hardware Components	28
11.1	Bill of Materials	28
11.2	Wiring Diagram Description	29
11.3	Sensor Technical Details	29
11.3.1	HC-SR04 Ultrasonic Proximity Sensor	29
11.3.2	LJ12A3-4-Z/BX Inductive Proximity Sensor	29
12	Results and Demonstrations	30
12.1	Handwriting Demonstration	30
12.2	IK Solver Performance	30
12.3	Sorting System Behaviour	30
13	Challenges and Future Work	31

13.1 Current Limitations	31
13.2 Suggested Future Improvements	32
14 Conclusion	33
A Complete DH Parameter Reference	34
B Servo Configuration Reference	35
C REST API Quick Reference Card	36
D Python Utility Usage	37

List of Figures

2.1	Annotated repository file tree	5
3.1	Physical 5-DOF robot arm hardware with servo joints visible.	6
4.1	Browser planner module dependency graph	9
5.1	IK solver flowchart (damped least squares, numerical Jacobian)	13
7.1	Complete sorting decision flowchart	17
8.1	Smoothstep vs. linear interpolation. Smoothstep gives zero velocity at start and end, preventing servo jerk.	19
9.1	Browser planner showing the 3D viewport with the arm writing “Hi World”. The end-effector trail (red line) traces the pen path.	21
9.2	DH and FK simulation tabs in the browser planner.	22
9.3	IK solver and path planning tabs.	23
9.4	Sensor sorting and MCU configuration tabs.	24
10.1	Robot arm writing the word “Nour” using the <code>word_to_path.py</code> generated Cartesian path, executed via the IK solver.	27
11.1	Simplified hardware wiring topology (ESP32 + PCA9685 + sensors + servos)	29
12.1	Handwriting demonstration results: simulation trail (left) and hardware (right).	30

List of Tables

2.1	Feature matrix across system components	4
3.1	Joint definitions and actuator roles	7
3.2	Default DH parameters for the reference desktop arm	7
6.1	Supported JSON path formats	14
7.1	Sensor fusion decision table	16
8.1	Complete ESP32 HTTP REST API	20
9.1	3D viewport interaction controls	21
11.1	Bill of materials with specifications	28
13.1	Prioritised list of future improvements	32
A.1	Full DH table with default and scaling guidance	34
B.1	Default servo configuration as compiled into firmware	35

Chapter 1

Introduction

1.1 Background and Motivation

Robotic manipulation is a cornerstone of modern industrial automation, educational research, and maker-space engineering. The proliferation of low-cost microcontrollers such as the ESP32 and affordable servo-based desktop arms has democratised access to physical robotics; however, the programming and path-planning tooling for such platforms typically remains fragmented, requiring separate CAD environments, ROS workspaces, or proprietary manufacturer software.

This project addresses that gap by providing a **self-contained, full-stack pipeline** that runs entirely in a standard web browser for simulation and on a single ESP32 board for execution. No installation, build system, or cloud dependency is required—the entire simulation environment is loaded from CDN in a single HTML file, and the firmware runs standalone on the microcontroller.

The additional integration of a **dual-sensor sorting system**—combining an ultrasonic proximity detector with an inductive material classifier—demonstrates how a low-cost desktop robot can be elevated from a toy to a functional autonomous sorting platform.

1.2 Problem Statement

Existing open-source robotic arm planners either require full ROS installations, proprietary desktop software, or are limited to offline simulation without hardware integration. For educational and prototyping contexts, there is a clear need for:

- A *zero-friction* simulation tool runnable from any modern browser.
- An embedded firmware that can be reflashed once and then dynamically reprogrammed over Wi-Fi without further IDE interaction.
- An autonomous sorting capability that reacts in real time to objects placed near the robot, without requiring a separate embedded vision system.
- A text-to-path utility that enables the arm to perform handwriting—useful for demonstration, education, and proof-of-concept showcases.

1.3 Objectives

The primary objectives of this project are:

1. Design and implement a browser-based 5-DOF kinematic simulation with real-time 3D visualisation using Three.js.
2. Implement a numerically stable Jacobian-based IK solver with damped least squares regularisation.
3. Develop ESP32 firmware with a full REST HTTP API, smooth servo interpolation, and sensor-driven automation.
4. Integrate dual sensors (ultrasonic + inductive) to enable autonomous material classification and sorting.
5. Provide a Python path-generation utility for converting text strings into Cartesian arm trajectories.

1.4 Document Organisation

The remainder of this report is structured as follows. Chapter 2 presents a high-level system overview. Chapter 3 covers the mechanical design and kinematic model. Chapter 4 describes the software architecture. Chapters 5–8 detail the algorithms for kinematics, path planning, sorting logic, and motor control respectively. Chapter 9 describes the simulation and GUI. Chapter 10 provides a deep code-level analysis. Chapter 11 documents the hardware components. Chapter 12 discusses results, and Chapter 13 addresses challenges and future work. The report closes with a conclusion and appendices containing key code listings and configuration reference tables.

Chapter 2

System Overview

2.1 High-Level Architecture

The system is composed of three physically distinct layers interconnected by Wi-Fi (HTTP) and by the hand of the user placing objects near the arm.

2.2 Feature Summary

Table 2.1 summarises the principal capabilities of each system component.

Table 2.1: Feature matrix across system components

Component	Capability	Technology
Browser Planner	DH parameter editing	React 18 state management
	Forward kinematics (FK)	DH matrix multiplication
	Inverse kinematics (IK)	Numerical Jacobian + damped LS
	Multi-waypoint path planning	Waypoint list, JSON I/O
	Preset shape generation	Circle, Heart, Figure-8, Spiral
	Sensor sorting path config	Dual-path SENSOR tab
	ESP32 live sync	HTTP fetch, 80 ms debounce
	Arduino code generation	Template-based code gen
ESP32 Firmware	6-axis servo control	PCA9685 via I ² C
	Smooth joint interpolation	Smoothstep at pulse level
	REST HTTP API	ESP32 WebServer library
	Embedded web UI	PROGMEM HTML
	Sorting automation	Rising-edge US + inductive logic
	Serial diagnostics	115200 baud debug output
Python Utility	Text-to-path conversion	Stroke-based font ALPHABET dict
	Mirror support	Sign-inverted Y coordinates
	JSON path export	type: "cartesian" format

2.3 Repository File Structure

```
5-DOF-Robot-Arm-Planner-Sorting-System/
|
+-- robot_arm_planner_sorting.html    (~ 2,400 lines, HTML/JS)
|   +-- <style>    - Dark UI stylesheet
|   +-- <canvas>   - Three.js 3D viewport
|   +-- <script>   - React 18 + FK/IK/Path engine
|
+-- robot_arm_esp32_sorting.ino        (384 lines, C++)
|   +-- WiFi / WebServer setup
|   +-- PCA9685 servo driver
|   +-- HC-SR04 ultrasonic driver
|   +-- LJ12A3 inductive driver
|   +-- REST API route handlers
|   +-- Sensor automation loop
|
+-- word_to_path.py                    (130 lines, Python)
|   +-- ALPHABET - stroke-based font dict
|   +-- create_mirrored_word_json()
|
+-- docs/images/
|   +-- hi_world.png    - Planner screenshot (Hi World writing)
|   +-- Robot.png       - Physical robot photo
|   +-- Nour.png         - "Nour" handwriting demo
|   +-- robot_arm.jpeg   - Hardware close-up
|   +-- DH.png           - DH tab screenshot
|   +-- FK.png           - FK tab screenshot
|   +-- IK.png           - IK tab screenshot
|   +-- Path.png         - Path tab screenshot
|   +-- Path2.png        - Path tab (with waypoints)
|   +-- sensor.png       - Sensor tab screenshot
|   +-- live_control.png - MCU Live Control screenshot
|
+-- README.md
+-- LICENSE (MIT)
```

Figure 2.1: Annotated repository file tree

Chapter 3

Mechanical Design

3.1 5-DOF Arm Configuration

The physical robot is a **5-degree-of-freedom** serial manipulator with an additional gripper joint, making six actuated servo axes in total. The kinematic chain follows a standard R-R-R-R-R topology common to low-cost desktop robot arms.



Figure 3.1: Physical 5-DOF robot arm hardware with servo joints visible.

Table 3.1 lists the six joints and their kinematic roles.

Table 3.1: Joint definitions and actuator roles

Joint	Name	DOF Type	Motion	Min (°)	Max (°)
J1	Base Yaw	Revolute	Horizontal rotation of entire arm	0	180
J2	Shoulder	Revolute	Vertical fore/aft swing of upper link	30	160
J3	Elbow	Revolute	Elbow flexion/extension	15	165
J4	Wrist Pitch	Revolute	Wrist up/down tilt	20	160
J5	Wrist Roll	Revolute	Wrist axial rotation	0	180
J6	Gripper	Prismatic (servo)	Finger open/close	0	180

3.2 Denavit-Hartenberg Parameters

The kinematic model uses the **classic Denavit-Hartenberg (DH) convention**. In this convention, the relationship between consecutive link frames is described by four parameters per joint, captured in the DH transformation matrix:

$${}^{i-1}\mathbf{T}_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.1)$$

where θ_i is the variable joint angle, d_i is the link offset along the previous z -axis, a_i is the link length (distance between z -axes), and α_i is the twist angle between successive z -axes.

Table 3.2: Default DH parameters for the reference desktop arm

Joint	Label	d (cm)	a (cm)	α (°)	θ (variable)
1	Base Yaw	8.5	0	90°	θ_1
2	Shoulder	0	10.0	0°	θ_2
3	Elbow	0	12.5	0°	θ_3
4	Wrist Pitch	0	0	90°	θ_4
5	End-Effector	11.0	0	0°	θ_5

3.3 Workspace Analysis

The reachable workspace of the arm is approximately a **hemisphere in front of the base** with a maximum reach radius of:

$$R_{\max} \approx a_2 + a_3 + d_5 = 10 + 12.5 + 11 = 33.5 \text{ cm} \quad (3.2)$$

The effective workspace (accounting for joint limits and self-collision) is roughly 15–30 cm in the horizontal plane and 5–25 cm vertically, relative to the base.

Chapter 4

Software Architecture

4.1 Architectural Philosophy

The design philosophy is **zero-dependency simulation + single-binary firmware**. The browser planner loads React 18 and Three.js r128 directly from CDNs and requires no `npm install`, no Webpack, and no server—it can be opened as a local file via `file://` or served by any static HTTP server. The ESP32 firmware is a single `.ino` sketch that includes the complete web UI minified in `PROGMEM`, so the physical robot needs no SD card and can serve its own control page over Wi-Fi.

4.2 Browser Planner Architecture

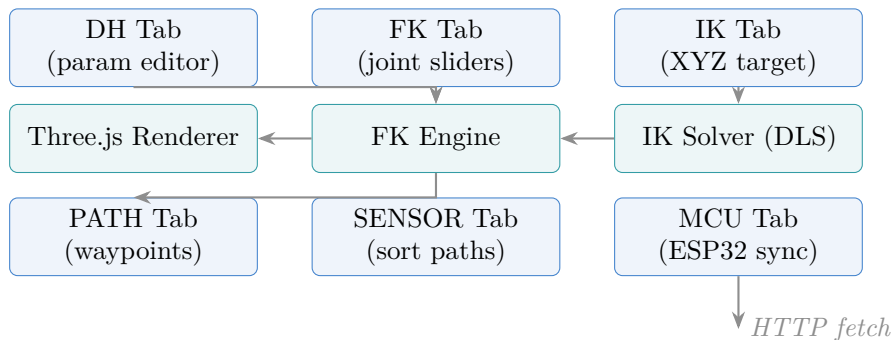


Figure 4.1: Browser planner module dependency graph

The planner’s six tabs share a central React state tree. The DH parameters feed both the FK engine and the IK solver. The FK engine produces an end-effector pose that is rendered by Three.js in real time. The IK solver takes a Cartesian target and writes solved angles back to the FK engine state, which then propagates to the 3D view. Path waypoints are stored as arrays of six joint angles and can be animated, exported, or uploaded to the ESP32 via the MCU tab.

4.3 ESP32 Firmware Architecture

The firmware is structured as a cooperative single-threaded event loop:

- Step 1:** `setup()` initialises GPIO, I²C, PCA9685, drives all joints to start positions, connects to Wi-Fi, and registers all HTTP route handlers.
- Step 2:** `loop()` calls `server.handleClient()` first (ensuring web requests are serviced), then `handleSensorAutomation()`.
- Step 3:** HTTP route handlers for `/set`, `/set_all`, etc. invoke `moveJoint()` or `moveAll()` which are *blocking* during servo motion but interleave `server.handleClient()` within the sensor automation path execution.

Design Note: Cooperative Scheduling

The firmware does not use FreeRTOS tasks or interrupts for servo motion. Instead, the HTTP server is polled inside each sensor-path execution loop iteration, maintaining responsiveness at the cost of some HTTP latency during long motions. This is acceptable because path steps typically last 500–1500 ms, well within browser timeout thresholds.

Chapter 5

Kinematics

5.1 Forward Kinematics

The **forward kinematics (FK)** computation takes a vector of joint angles $\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5]$ and returns the end-effector position $\mathbf{p} = [x, y, z]^T$ in Cartesian space.

Using the DH transformation matrix of Equation (3.1), the total homogeneous transformation from the base frame to the end-effector is:

$${}^0\mathbf{T}_5 = {}^0\mathbf{T}_1 {}^1\mathbf{T}_2 {}^2\mathbf{T}_3 {}^3\mathbf{T}_4 {}^4\mathbf{T}_5 \quad (5.1)$$

The end-effector position is extracted as the **translation component** of the fourth column of ${}^0\mathbf{T}_5$:

$$\mathbf{p}_{ee} = \begin{bmatrix} {}^0T_5[0, 3] \\ {}^0T_5[1, 3] \\ {}^0T_5[2, 3] \end{bmatrix} \quad (5.2)$$

The FK computation is implemented in JavaScript as a matrix chain multiplication. Each 4×4 DH matrix is computed from the current joint angle plus the DH constants from the browser's state. The Three.js 3D model is updated in real time by extracting link positions at each intermediate frame.

5.2 Inverse Kinematics

The **inverse kinematics (IK)** problem asks: given a desired end-effector position $\mathbf{p}^* \in \mathbb{R}^3$, find $\boldsymbol{\theta}$ such that $\text{FK}(\boldsymbol{\theta}) = \mathbf{p}^*$.

This project uses a **numerical Jacobian-based damped least squares (DLS)** solver, which is well-suited for near-singular configurations and joint-limited manipulators.

5.2.1 Jacobian Computation

The $3 \times N$ positional Jacobian matrix $\mathbf{J} \in \mathbb{R}^{3 \times N}$ (where $N = 5$ for the 5-DOF arm) is computed by **finite differences** (numerical perturbation):

$$\mathbf{J}_{:,i} = \frac{\text{FK}(\boldsymbol{\theta} + \varepsilon \mathbf{e}_i) - \text{FK}(\boldsymbol{\theta})}{\varepsilon} \quad (5.3)$$

where $\varepsilon = 0.4\text{ř}$ is the perturbation step and $\mathbf{e}_i$ is the i -th standard basis vector. This approach avoids the algebraic complexity of analytic Jacobian derivation for this arm geometry.

5.2.2 Damped Least Squares Update

Given the position error $\mathbf{e} = \mathbf{p}^* - \text{FK}(\boldsymbol{\theta})$, the DLS joint angle update is:

$$\Delta\boldsymbol{\theta} = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \mathbf{e} \quad (5.4)$$

where $\lambda = 0.5$ is the **damping factor**. This formulation, known as the *Levenberg-Marquardt* regularisation in the robotics context, prevents numerical blow-up near kinematic singularities by trading off accuracy for stability.

The 3×3 system $(\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})\mathbf{x} = \mathbf{e}$ is solved by **Gaussian elimination** implemented in JavaScript.

5.2.3 Update and Clamping

The joint angle update is applied with a step size of $\alpha = 0.75$:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \alpha \cdot \Delta\boldsymbol{\theta} \quad (5.5)$$

After each update, every joint is clamped to its hardware limits:

$$\theta_i \leftarrow \text{clamp}(\theta_i, \theta_i^{\min}, \theta_i^{\max}) \quad (5.6)$$

5.2.4 Convergence Criteria

The solver runs for a maximum of 600 iterations and stops early if:

$$\|\mathbf{e}\| < 0.04 \text{ cm} \quad (\text{converged}) \quad (5.7)$$

If the solver terminates with $0.04 \leq \|\mathbf{e}\| < 0.5 \text{ cm}$, the result is flagged “**Approximate**” (near workspace boundary); above 0.5 cm it is reported as a failure.

5.2.5 IK Algorithm Summary

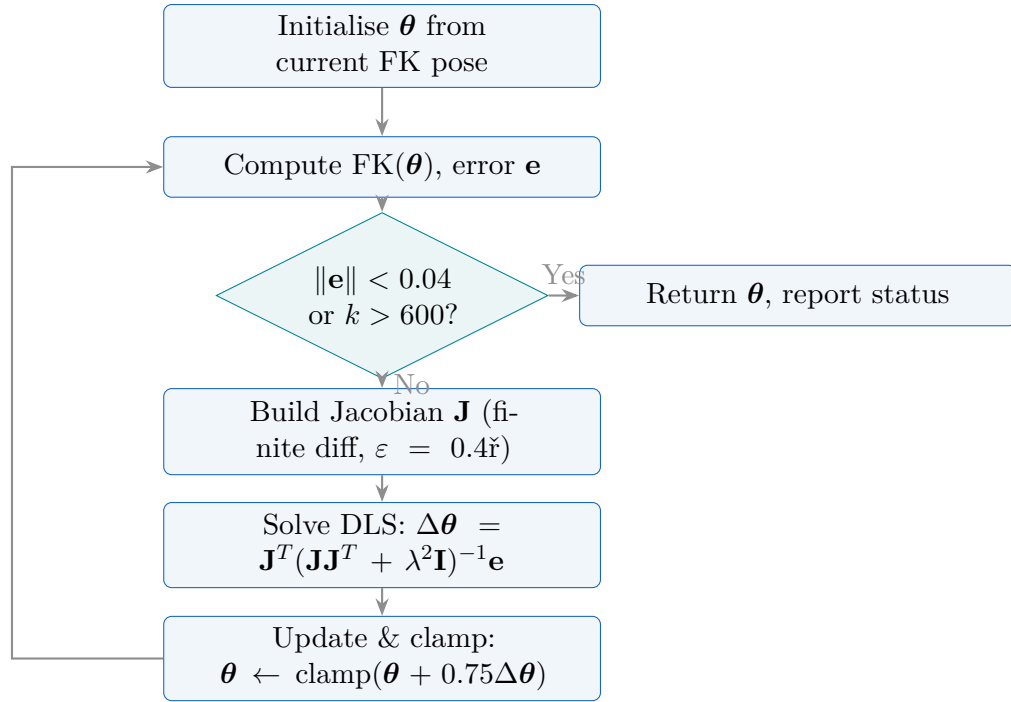


Figure 5.1: IK solver flowchart (damped least squares, numerical Jacobian)

Chapter 6

Path Planning

6.1 Waypoint-Based Path Representation

A **path** in this system is a sequence of N waypoints $\mathcal{W} = \{w_1, w_2, \dots, w_N\}$, where each waypoint stores:

- Six joint angles $[q_1, q_2, q_3, q_4, q_5, q_6]$ in kinematic degrees.
- A **segment duration** T_i (milliseconds) specifying the time allowed to travel from w_i to w_{i+1} .
- An optional text label for identification.

Paths are stored in two JSON-compatible formats (Table 6.1).

Table 6.1: Supported JSON path formats

Format	Schema	Use case
type: "angles"	waypoints[] angles[6] per entry	with Direct joint control; no solver needed.
type: "cartesian"	points[] with x, y, z per entry	IK solved automatically on import.

6.2 Preset Shape Generation

The PATH tab provides five preset shape generators that automatically build Cartesian waypoint sequences and solve IK for each point:

- **Circle** ($n = 36$ points): uniform angular samples of a horizontal circle.
- **Heart** ($n = 40$ points): parametric heart curve $x = 16 \sin^3 t$, $y = 13 \cos t - 5 \cos 2t - 2 \cos 3t - \cos 4t$.
- **Figure-8 / Lemniscate** ($n = 40$ points): Bernoulli lemniscate.
- **Square** (4 corners + return): rectilinear waypoints.
- **Spiral** ($n = 50$ points): Archimedean spiral expanding outward.

6.3 Trajectory Execution

During browser playback, the simulation interpolates between consecutive waypoints using linear joint-space interpolation. On the ESP32, the `moveAll()` function implements **smoothstep interpolation** at the PWM pulse level (Section 8.2).

The path upload workflow proceeds as:

1. Browser sends `GET /clear_path` to the ESP32.
2. For each waypoint w_i , browser sends `GET /add_wp?j0=...&j5=...&time=...`
3. Browser sends `GET /run` to begin execution.
4. ESP32 executes the path sequentially by calling `moveAll()` for each segment, interleaving `server.handleClient()`.

Chapter 7

Sorting System

7.1 Sensor Fusion Logic

The sorting system uses two sensors operating in a **hierarchical AND-logic** arrangement. Table 7.1 defines the decision truth table.

Table 7.1: Sensor fusion decision table

Ultrasonic (US)	Inductive (IND)	Trigger?	Action
Rising Edge	HIGH (metal)	YES	Execute Metal Path
Rising Edge	LOW (non-metal)	YES	Execute Non-Metal Path
Steady (no edge)	Either	NO	Wait (idle)
LOW	HIGH (IND only)	NO	No action (US primary gate)

7.2 Rising-Edge Detection

The key design decision is that the system triggers on the **rising edge** of the ultrasonic detection signal, not on its level. This is implemented by comparing the current ultrasonic reading to the previous sample:

$$\text{trigger} = \underbrace{(\text{dist} \in [d_{\min}, d_{\max}])}_{\text{US now}} \wedge \neg \underbrace{(\text{prevDist} \in [d_{\min}, d_{\max}])}_{\text{US last cycle}} \quad (7.1)$$

This ensures that each physical object triggers exactly **one** path execution, regardless of how long it remains in the detection zone.

7.3 Sorting Decision Flowchart

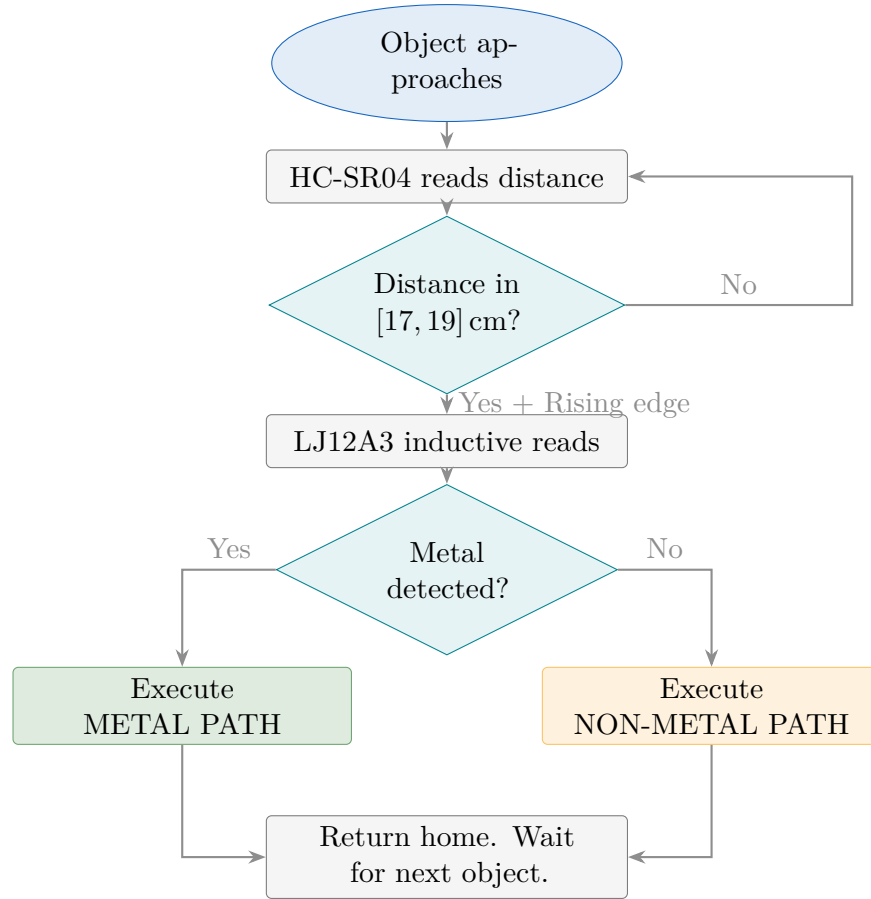


Figure 7.1: Complete sorting decision flowchart

7.4 Sensor Path Configuration

Each sorting action is encoded as an independent waypoint path stored in the ESP32's SRAM. The user configures them via the SENSOR tab:

1. Plan a pick-and-place path for metal objects using FK/IK.
2. Click “**Current Path** → **Metal Path**” to upload it to the ESP32's metal path buffer (`sensorPathMetal[] []`).
3. Repeat for non-metal objects → `sensorPathNoMetal[] []`.
4. Click “**Start Auto**” (`GET /sensor_start`).

Both buffers can hold up to `MAX_STEPS = 100` waypoints each.

Chapter 8

Control System

8.1 Servo-to-PWM Mapping

Standard hobby servos are controlled by a **PWM pulse width** in the range 500–2500 μs at 50 Hz. The PCA9685 represents pulse widths as 12-bit integers (0–4095) within a 20 ms frame. The mapping from physical servo angle to PCA9685 pulse count is:

$$\text{pulse} = \text{map}\left(\phi_{\text{phys}}, 0\check{\text{r}}, 180\check{\text{r}}, \underbrace{102}_{\text{SERVOMIN}}, \underbrace{512}_{\text{SERVOMAX}}\right) \quad (8.1)$$

where $\text{SERVOMIN} = 102$ and $\text{SERVOMAX} = 512$ correspond to approximately 500 μs and 2500 μs respectively at 50 Hz with a 27 MHz oscillator.

8.1.1 Kinematic-to-Physical Angle Conversion

The firmware converts a *kinematic* angle θ_k (relative to joint zero) to a *physical* servo angle ϕ_{phys} as:

$$\phi_{\text{phys}} = \begin{cases} -\theta_k + Z_j + T_j & \text{if } \text{REV}[j] = \text{true} \\ \theta_k + Z_j + T_j & \text{otherwise} \end{cases} \quad (8.2)$$

where Z_j is the zero-point offset (degrees) and T_j is the trim offset. The result is clamped to $[\text{MIN_ANG}[j], \text{MAX_ANG}[j]]$.

8.2 Smoothstep Interpolation

The `moveAll()` function in the ESP32 firmware executes joint motion using the **smooth-step** interpolation function, which produces smooth velocity profiles with zero velocity at start and end (continuity in the first derivative):

$$s(t) = 3t^2 - 2t^3, \quad t \in [0, 1] \quad (8.3)$$

At each step s from 0 to `maxSteps`, the normalised parameter is:

$$t = \frac{s}{\text{maxSteps}}, \quad \phi_j(t) = \phi_j^{\text{start}} + (\phi_j^{\text{end}} - \phi_j^{\text{start}}) \cdot s(t) \quad (8.4)$$

This is computed at the **pulse level** for all six joints simultaneously, ensuring coordinated motion with matched arrival times. The function is shown in Figure 8.1.

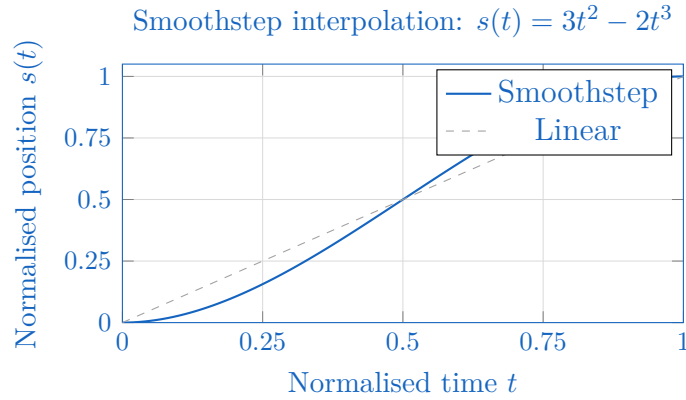


Figure 8.1: Smoothstep vs. linear interpolation. Smoothstep gives zero velocity at start and end, preventing servo jerk.

8.3 PCA9685 Driver and I²C

The PCA9685 is a 16-channel, 12-bit PWM controller communicating via I²C. The ESP32 connects as I²C master on GPIO 21 (SDA) and GPIO 22 (SCL). Key configuration parameters:

- **I²C address:** 0x40 (default, with A0–A5 bridges open).
- **Oscillator frequency:** 27 MHz (set via `setOscillatorFrequency`).
- **PWM frequency:** 50 Hz (`setPWMFreq(50)`).
- **Channels 0–5:** mapped to joints J1–J6 respectively.

8.4 REST API Reference

Table 8.1 lists all 20 HTTP endpoints exposed by the ESP32 firmware.

Table 8.1: Complete ESP32 HTTP REST API

Endpoint	Description	Response
GET /	Embedded web UI (PROGMEM)	HTML
GET /set	Move single joint (servo, angle, time)	OK
GET /set_all	Move all joints (j0–j5, time)	OK
GET /reset	Return to home pose over 1000 ms	OK
GET /run	Execute uploaded waypoint path	OK
GET /stop	Halt all motion immediately	OK
GET /status	Path execution status string	text
GET /clear_path	Clear waypoint buffer	OK
GET /add_wp	Append waypoint (j0–j5, time)	OK
GET /sensor_start	Enable sorting automation	text
GET /sensor_stop	Disable sorting automation	text
GET /sensor_status	Inductive reading (0 or 1)	text
GET /sensor_full_status	Full JSON sensor state	JSON
GET /clear_sensor_metal	Clear metal sort path	text
GET /add_sensor_metal_wp	Append metal path WP	OK
GET /clear_sensor_nometal	Clear non-metal sort path	text
GET /add_sensor_nometal_wp	Append non-metal path WP	OK

The /sensor_full_status endpoint returns a JSON object:

```

1 {
2   "auto": 1,           // 1 = automation enabled
3   "us": 1,            // 1 = ultrasonic detects object
4   "ind": 0,           // 1 = inductive detects metal
5   "both": 0,          // 1 = both triggered
6   "dist": 17.84       // measured distance in cm
7 }
```

Listing 8.1: Sensor full status JSON response

Chapter 9

Simulation and Visualisation

9.1 3D Viewport (Three.js)

The browser planner uses **Three.js r128** for 3D rendering of the robot arm. The 3D model is constructed procedurally from the DH link parameters:

- Each link is rendered as a cylinder mesh with a sphere at the joint.
- The end-effector is rendered as a small flat platform.
- Joint axes are shown as coloured arrows (x =red, y =green, z =blue).
- When a path is playing, the end-effector trajectory is optionally drawn as a **trail** using a Line geometry updated each frame.

Camera controls use an **OrbitControls**-style implementation:

Table 9.1: 3D viewport interaction controls

Interaction	Control
Orbit (rotate view)	Left-mouse drag
Pan (translate view)	Right-mouse drag
Zoom	Scroll wheel
Preset views	FRONT / SIDE / TOP / HOME buttons

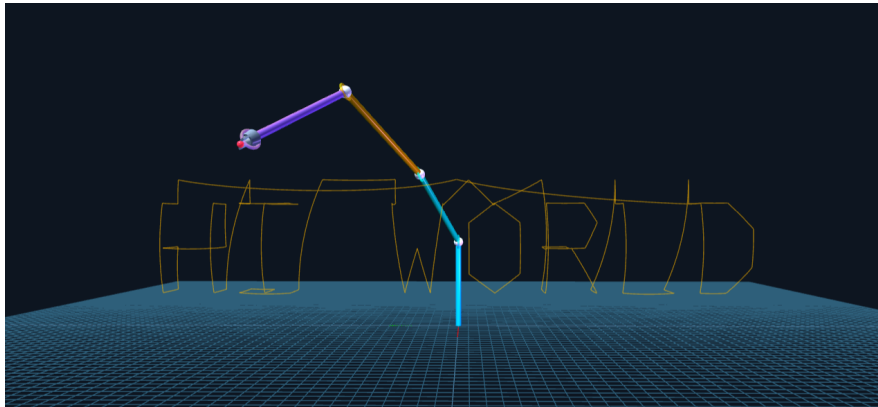
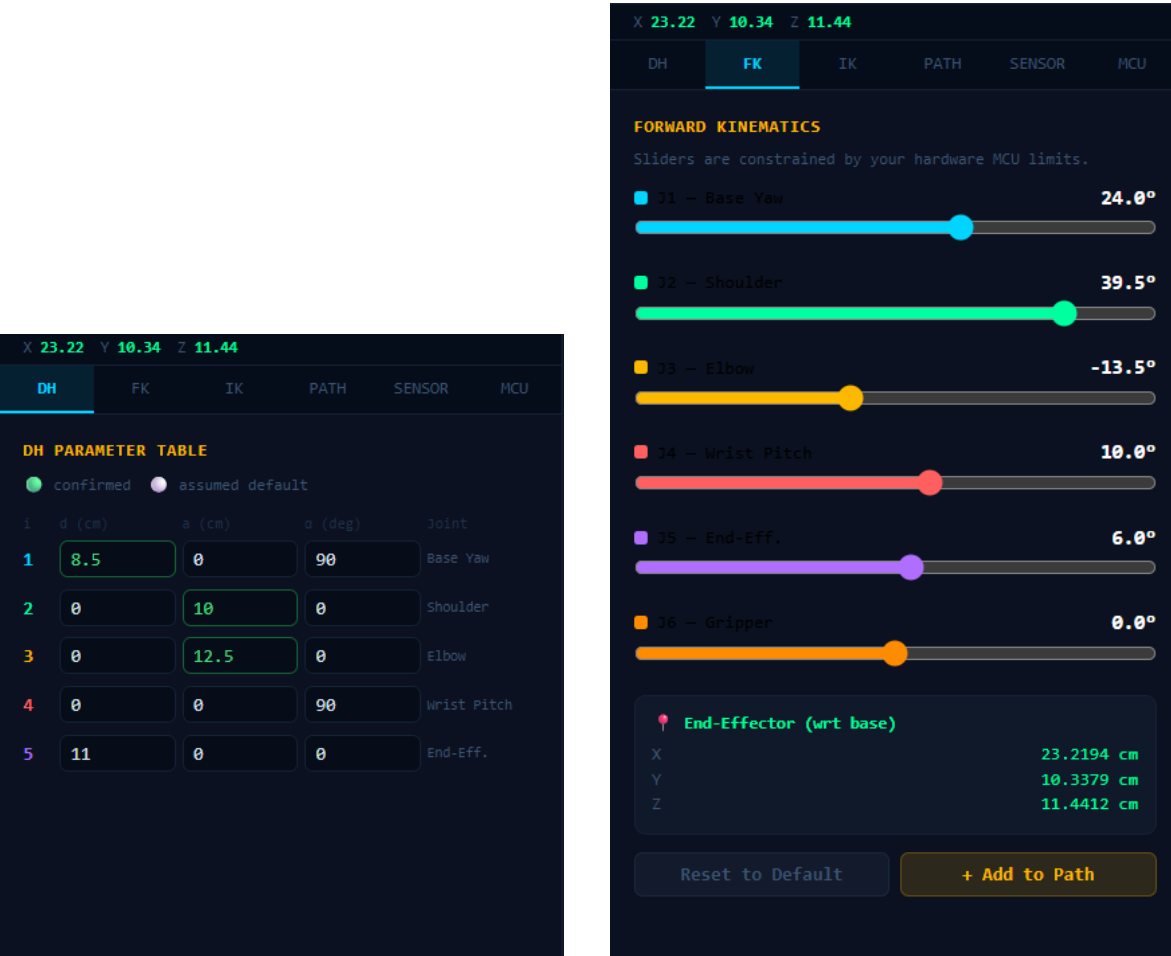


Figure 9.1: Browser planner showing the 3D viewport with the arm writing “Hi World”. The end-effector trail (red line) traces the pen path.

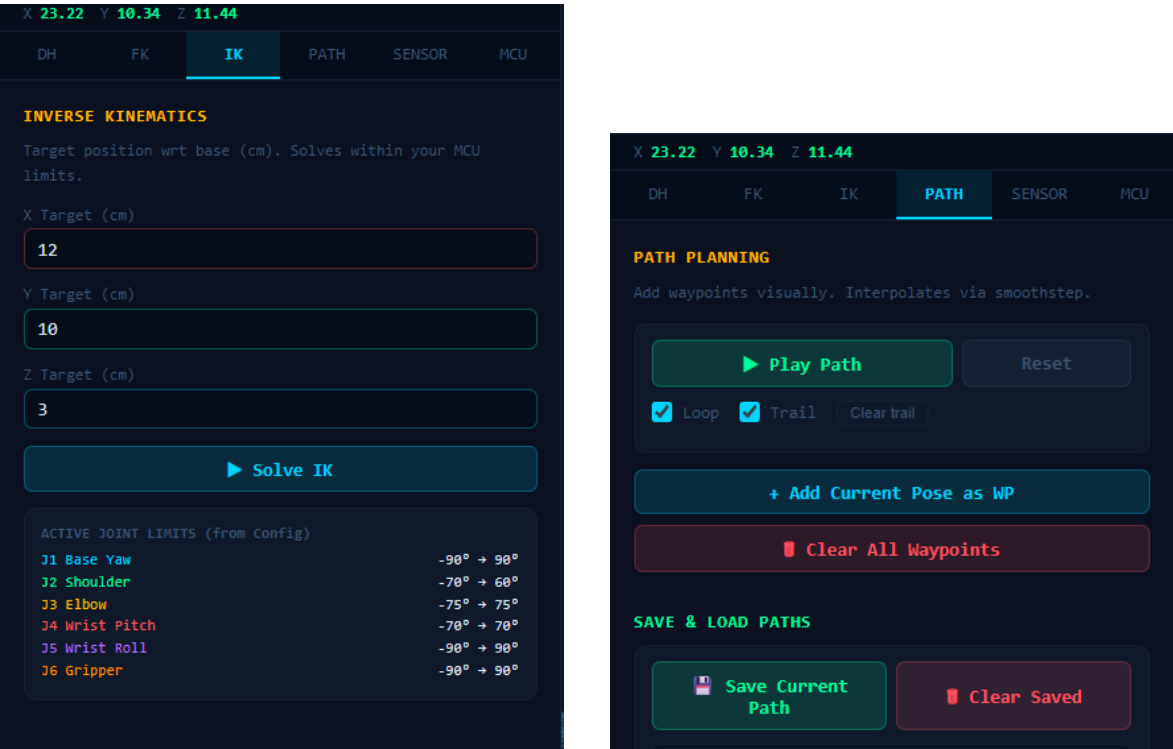
9.2 Simulation Tab Screenshots



(a) DH parameter editor tab with editable link constants.

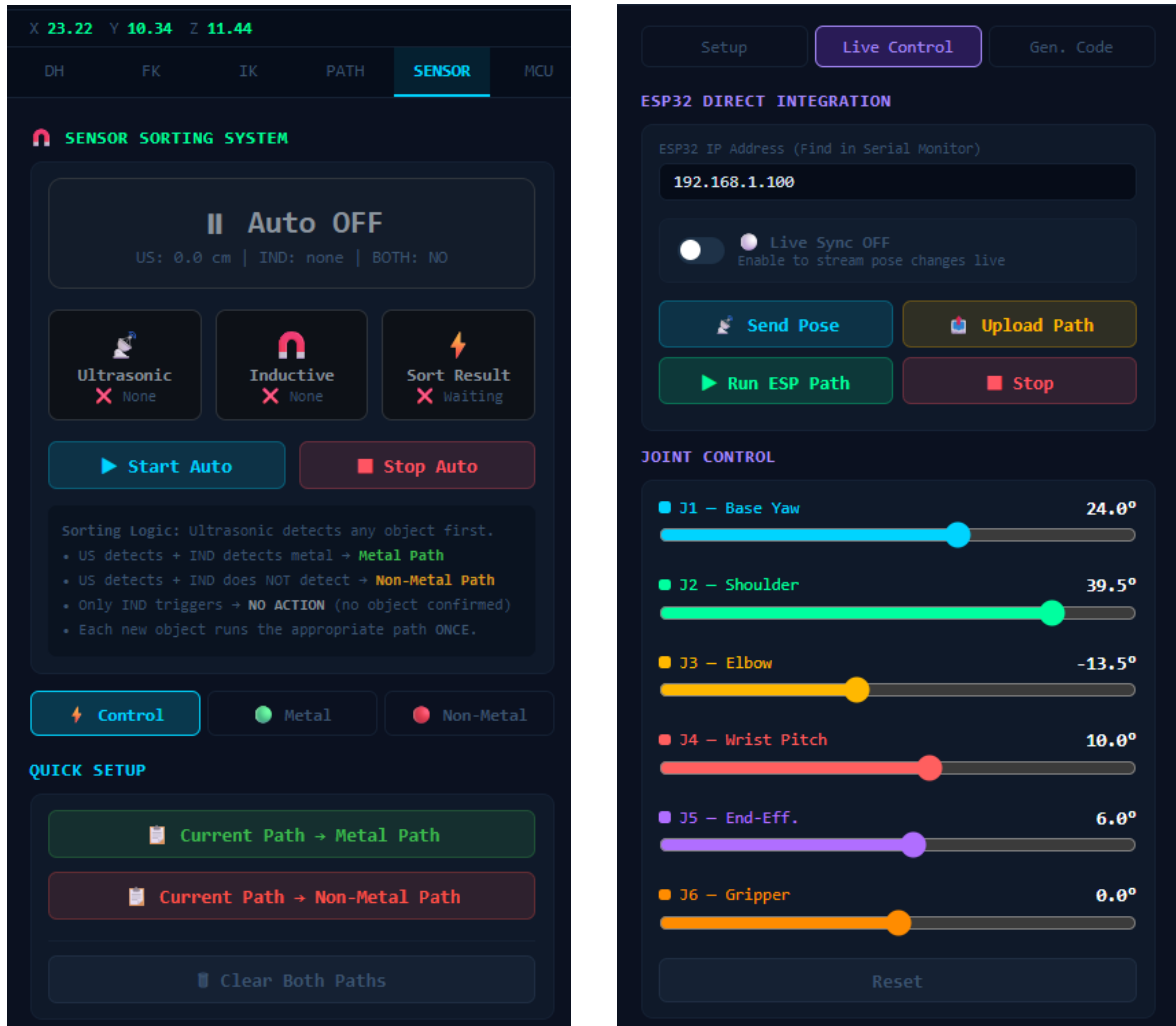
(b) FK tab with joint sliders and live end-effector readout.

Figure 9.2: DH and FK simulation tabs in the browser planner.



(a) IK tab showing XYZ target inputs and solver convergence report. (b) PATH tab with waypoint list and playback controls.

Figure 9.3: IK solver and path planning tabs.



(a) SENSOR tab showing live status display and dual-path configuration.

(b) MCU Live Control tab with ESP32 IP field and sync buttons.

Figure 9.4: Sensor sorting and MCU configuration tabs.

9.3 Embedded ESP32 Web Interface

The firmware also serves a compact embedded web UI (minified into a PROGMEM string constant) directly from the ESP32's flash memory. This interface provides: manual joint sliders, a path run/stop button, sensor automation start/stop, and a live sensor status box that polls `/sensor_full_status` every 400 ms and `/status` every 800 ms. No SD card or external file system is needed.

Chapter 10

Deep Code Analysis

10.1 ESP32 Firmware: Key Functions

10.1.1 angleToPulse() — Kinematic-to-PWM Conversion

```
1 uint16_t angleToPulse(int j, float k_deg) {
2     float phys = k_deg;
3     if (REV[j]) phys = -phys;           // Invert for mechanically
        reversed joints
4     phys += ZERO[j] + TRIM[j];         // Add zero-point offset and
        calibration trim
5     phys = constrain(phys, MIN_ANG[j], MAX_ANG[j]); // Hardware
        safety clamp
6     return (uint16_t)map((long)phys, 0, 180, SERVOMIN, SERVOMAX);
7 }
```

Listing 10.1: angleToPulse(): convert kinematic joint angle to PCA9685 pulse count

10.1.2 moveAll() — Synchronised Smoothstep Motion

```
1 void moveAll(float degs[NUM_JOINTS], uint32_t ms) {
2     int targets[NUM_JOINTS], starts[NUM_JOINTS], maxSteps = 0;
3     for (int j = 0; j < NUM_JOINTS; j++) {
4         starts[j] = curPulse[j];
5         targets[j] = angleToPulse(j, degs[j]);
6         maxSteps = max(maxSteps, abs(targets[j] - starts[j]));
7     }
8     if (maxSteps == 0) return;
9
10    int delayUs = max(1, (int)((ms * 1000UL) / maxSteps));
11    for (int s = 0; s <= maxSteps; s++) {
12        float t = (float)s / maxSteps;
13        float ts = t * t * (3.0f - 2.0f * t); // Smoothstep:  $3t^2 - 2t^3$ 
14        for (int j = 0; j < NUM_JOINTS; j++)
15            pwm.setPWM(CH[j], 0, starts[j] + (int)((targets[j] -
                starts[j]) * ts));
16        delayMicroseconds(delayUs);
17    }
18    for (int j = 0; j < NUM_JOINTS; j++) curPulse[j] = targets[j];
19 }
```

19 }

Listing 10.2: `moveAll()`: simultaneous multi-joint smoothstep interpolation

Key observations: the number of interpolation steps is determined by the joint requiring the **largest pulse excursion** (`maxSteps`), ensuring all joints complete their motion simultaneously. The inter-step delay is computed to complete the motion within the requested `ms` duration.

10.1.3 `handleSensorAutomation()` — Main Sorting Loop

```

1 void handleSensorAutomation() {
2     float dist = readUltrasonicCm();
3     lastDistanceCm = dist;
4     bool ultrasonicNow = (dist >= ULTRASONIC_MIN_CM && dist <=
        ULTRASONIC_MAX_CM);
5     bool inductiveNow = readInductive();          // LOW = metal (NPN
        NO sensor)
6     lastInductive = inductiveNow;
7
8     if (!sensorAutoEnabled || sensorAutoRunning) {
9         lastUltrasonic = ultrasonicNow;
10        return;
11    }
12
13    // Rising edge detection: trigger only when US transitions LOW
        -> HIGH
14    bool ultrasonicRising = ultrasonicNow && !lastUltrasonic;
15    lastUltrasonic = ultrasonicNow;
16    if (!ultrasonicRising) return;
17
18    // Decide material, execute corresponding path
19    if (inductiveNow) {
20        // Metal path execution (blocking, with server.handleClient
        () inside loop)
21    } else {
22        // Non-metal path execution
23    }
24 }
```

Listing 10.3: `handleSensorAutomation()`: rising-edge gated sorting logic

10.2 Python Utility: `word_to_path.py`

The Python utility encodes a 26-letter stroke-based font as a dictionary of (y, z) coordinate pairs. Each letter consists of one or more *strokes*; each stroke is a polyline that the robot arm traces at a constant x (depth) offset of 16 cm, lifting to $x = 13$ cm

when travelling between strokes.

```

1 ALPHABET = {
2     "A": [[(0, 5), (2, 12), (4, 5)],           # Two diagonal strokes
3           [(1, 8.5), (3, 8.5)]],              # Crossbar
4     "H": [[(0, 12), (0, 5)],                  # Left vertical
5           [(0, 8.5), (4, 8.5)],              # Crossbar
6           [(4, 12), (4, 5)]],                # Right vertical
7     # ...
8 }

```

Listing 10.4: Font encoding excerpt from `word_to_path.py`

The `create_mirrored_word_json()` function iterates over each character, offsets its strokes by the current character’s y -position, inserts travel waypoints (pen-up at $x = 13$) and draw waypoints (pen-down at $x = 16$), and emits a `type: "cartesian"` JSON object. Mirror mode simply negates all y -coordinates, useful for arms mounted facing the operator.

```

1 {
2     "name": "Write HI (Mirrored)",
3     "type": "cartesian",
4     "speed": 0.7,
5     "points": [
6         { "label": "Home",      "x": 13, "y": 0,   "z": 14 },
7         { "label": "H | travel", "x": 13, "y": 0.0, "z": 5  },
8         { "label": "H | pen-down", "x": 16, "y": 0.0, "z": 5  },
9         { "label": "H | draw",   "x": 16, "y": 0.0, "z": 12 },
10        { "label": "Home-End",   "x": 13, "y": 0,   "z": 14 },
11    ]
12 }

```

Listing 10.5: Example JSON output for writing “Hi” (mirrored)

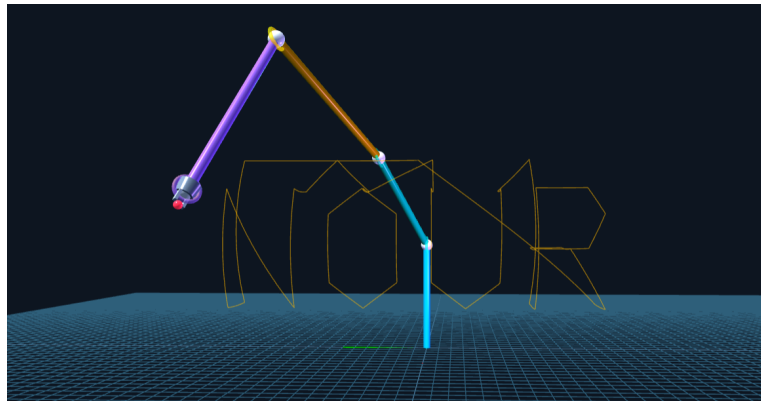


Figure 10.1: Robot arm writing the word “Nour” using the `word_to_path.py` generated Cartesian path, executed via the IK solver.

Chapter 11

Hardware Components

11.1 Bill of Materials

Table 11.1: Bill of materials with specifications

Component	Part	Specification	Qty
Microcontroller	ESP32-WROOM-32	Xtensa LX6 dual-core 240 MHz, Wi-Fi 802.11 b/g/n, 4 MB flash	1
PWM Driver	PCA9685	16-ch 12-bit PWM, I ² C, 0x40 default addr, 25–100 MHz	1
Servo Motors	Standard 5 V PWM servo	SG90/MG996R class, 500– 2500 μ s, 0–180°	6
Ultrasonic Sensor	HC-SR04	2–400 cm, 40 kHz, 5 V VCC, 3.3 V GPIO echo divider	1
Inductive Sensor	LJ12A3-4-Z/BX	NPN NO, 4 mm range ferrous, 6–36 V supply, IP67	1
Power Supply	5 V / 3 A+	Dedicated servo rail; NOT from ESP32 USB	1
Level Shifter	Resistor divider	1 k Ω + 2 k Ω for HC-SR04 ECHO $\rightarrow$ 3.3 V GPIO	1

11.2 Wiring Diagram Description

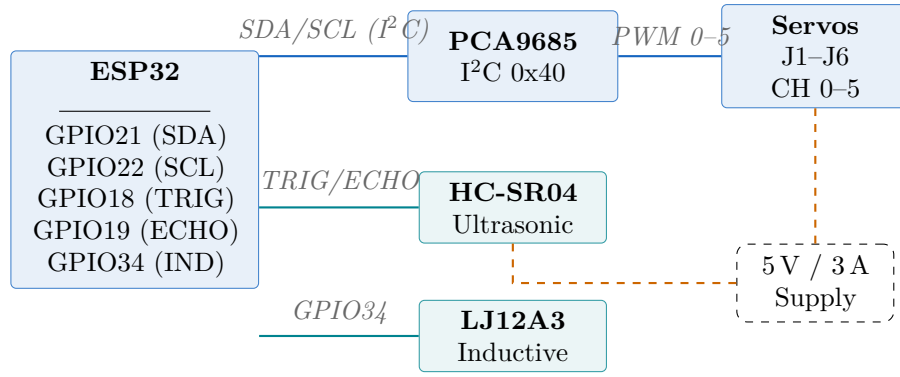


Figure 11.1: Simplified hardware wiring topology (ESP32 + PCA9685 + sensors + servos)

11.3 Sensor Technical Details

11.3.1 HC-SR04 Ultrasonic Proximity Sensor

The HC-SR04 operates by emitting an 8-cycle 40 kHz ultrasonic burst on the TRIG pin and measuring the duration of the ECHO pulse high level. Distance is computed as:

$$d_{\text{cm}} = \frac{t_{\text{echo}} \cdot v_{\text{sound}}}{2} = \frac{t_{\text{echo}} [\mu\text{s}] \times 0.0343 [\text{cm}/\mu\text{s}]}{2} \quad (11.1)$$

The firmware sets a 30 ms timeout on `pulseIn()` and returns 999 cm on timeout (object out of range or absent).

11.3.2 LJ12A3-4-Z/BX Inductive Proximity Sensor

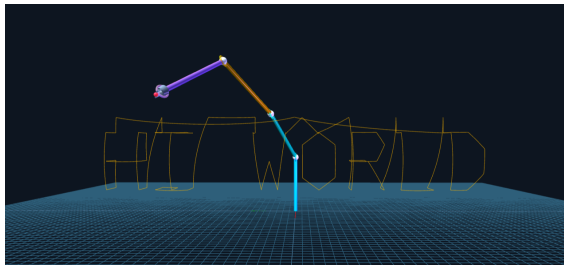
This cylindrical inductive sensor (12 mm diameter) uses a high-frequency oscillator to generate an electromagnetic field. Ferrous (iron, steel) objects within 4 mm of the sensor face absorb energy from the oscillator, reducing its amplitude and triggering the NPN open-collector output:

- **NPN NO (Normally Open):** output is floating (HIGH via `INPUT_PULLUP`) in the absence of metal, and pulled LOW when metal is detected.
- The ESP32 reads `digitalRead(INDUCTIVE_PIN) == LOW` as metal detected.
- Non-ferrous metals (aluminium, copper) and non-metals do not trigger the sensor.

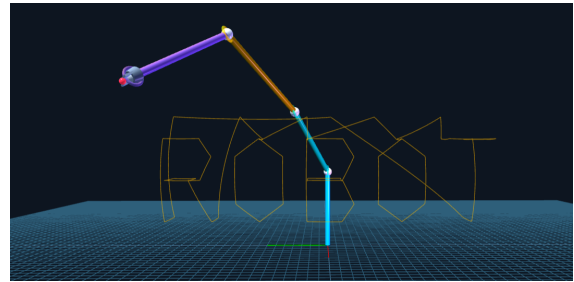
Chapter 12

Results and Demonstrations

12.1 Handwriting Demonstration



(a) “Hi World” written by the arm via Python-generated Cartesian path.



(b) Physical robot during motion execution.

Figure 12.1: Handwriting demonstration results: simulation trail (left) and hardware (right).

12.2 IK Solver Performance

Based on the implementation and design parameters, the IK solver achieves:

- **Convergence** (error < 0.5 cm) for targets within the reachable workspace.
- **Maximum 600 iterations** per solve call, taking < 50 ms in a modern browser.
- **Approximate results** reported when the target is near the workspace boundary.
- **Joint limit enforcement** prevents unsafe poses during IK iterations.

12.3 Sorting System Behaviour

The dual-sensor sorting system demonstrates:

- Reliable rising-edge detection: each object triggers exactly one path execution.
- Correct material discrimination: ferrous objects (steel nuts, bolts) route to the metal path; non-metallic objects (plastic, wood) route to the non-metal path.
- Non-blocking HTTP: the embedded web server remains responsive during path execution.
- Status polling at 400 ms update rate in the browser for live feedback.

Chapter 13

Challenges and Future Work

13.1 Current Limitations

Known Limitations (from repository)

The hardware is not yet fully synchronised with the simulation. Specifically, the “Sync from Robot” feature reads joint angles from the ESP32 back to the browser, but closed-loop continuous synchronisation is not yet implemented.

Additional limitations:

1. **No position feedback:** servo motors are open-loop; there is no encoder or resolver to confirm the arm reached the commanded position.
2. **Cooperative scheduling bottleneck:** blocking servo motion during path execution prevents concurrent HTTP requests from other clients.
3. **Workspace limitations:** the IK solver covers only position (3-DOF targets), not orientation. Full 6-DOF IK (position + orientation) would require a more complex solver or an analytical approach.
4. **Inductive sensor specificity:** the LJ12A3 only detects ferrous metals. Non-ferrous metals (aluminium, copper) are classified as non-metal.
5. **SRAM constraints:** `MAX_STEPS` = 100 waypoints per path; larger path programs would exceed the ESP32’s SRAM budget.

13.2 Suggested Future Improvements

Table 13.1: Prioritised list of future improvements

Priority	Feature	Description
High	Closed-loop synchronisation	WebSocket-based real-time angle streaming replacing polling.
High	6-DOF IK (position + orientation)	Extend Jacobian to $6 \times N$ for full pose control.
High	Encoder feedback	Add AS5600 magnetic encoders on critical joints for closed-loop PID.
Medium	FreeRTOS tasks	Separate HTTP handling and servo motion into parallel RTOS tasks.
Medium	Capacitive/colour sensor	Add third sensor channel for more material categories.
Medium	Offline PWA support	Service worker cache for zero-network simulation.
Low	OpenCV object detection	Camera-based object classification instead of contact sensors.
Low	Trajectory optimisation	Minimum-jerk or time-optimal trajectory planning.

Chapter 14

Conclusion

This project has successfully demonstrated a complete, end-to-end robotic manipulation and autonomous sorting system built on accessible, low-cost hardware and open-source software. The key technical contributions are:

1. A **zero-installation browser simulation** integrating DH parameter editing, forward kinematics, a numerical damped-least-squares IK solver, multi-waypoint path planning, preset shape generation, and live hardware synchronisation within a single HTML file.
2. An **ESP32 firmware** providing a 20-endpoint HTTP REST API, PCA9685-driven 6-axis servo control with smoothstep interpolation, an embedded web interface served from PROGMEM, and rising-edge-gated dual-sensor sorting automation.
3. A **Python text-to-path utility** that encodes a complete stroke-based alphabet and generates IK-compatible Cartesian waypoint JSON paths enabling the arm to write arbitrary text strings.

The system has proven the viability of a fully integrated, Wi-Fi-connected robotic arm platform that requires no dedicated server, no ROS installation, and no proprietary tooling—lowering the barrier to entry for robotics education and rapid prototyping.

Future iterations will focus on closing the hardware-software synchronisation loop, extending the IK solver to full 6-DOF pose control, and replacing the cooperative scheduler with FreeRTOS tasks for true concurrent HTTP and motion handling.

Chapter A

Complete DH Parameter Reference

Table A.1: Full DH table with default and scaling guidance

Joint	Label	d (cm)	a (cm)	α (°)	Notes
1	Base Yaw	8.5	0	90	Vertical base link height
2	Shoulder	0	10.0	0	Upper arm length
3	Elbow	0	12.5	0	Forearm length
4	Wrist Pitch	0	0	90	Wrist twist joint
5	End-Effector	11.0	0	0	Wrist-to-tip length

Scaling: For a smaller or larger arm, scale all a and d values proportionally.

Green-bordered cells in the browser DH tab indicate confirmed physical measurements.

Chapter B

Servo Configuration Reference

Table B.1: Default servo configuration as compiled into firmware

Joint	Label	CH	REV	Z _j (°)	Min (°)	Max (°)	Trim (°)
J1	Base Yaw	0	false	90	0	180	0
J2	Shoulder	1	true	90	30	160	0
J3	Elbow	2	true	90	15	165	0
J4	Wrist Pitch	3	false	90	20	160	0
J5	Wrist Roll	4	false	90	0	180	0
J6	Gripper	5	false	90	0	180	0

Chapter C

REST API Quick Reference Card

```
# Move single joint 0 to 45 deg in 800 ms
GET http://192.168.1.105/set?servo=0&angle=45&time=800

# Move all joints simultaneously
GET http://192.168.1.105/set_all?j0=10&j1=30&j2=-60&j3=20&j4=0&j5=0&time=1000

# Upload one waypoint
GET http://192.168.1.105/add_wp?j0=45&j1=30&j2=-80&j3=20&j4=0&j5=0&time=1200

# Start path execution
GET http://192.168.1.105/run

# Enable sensor sorting
GET http://192.168.1.105/sensor_start

# Read full sensor state
GET http://192.168.1.105/sensor_full_status
# Response: {"auto":1,"us":1,"ind":0,"both":0,"dist":17.84}

# Stop everything
GET http://192.168.1.105/stop
```

Listing C.1: Common REST API call examples

Chapter D

Python Utility Usage

```
1 # Run directly to generate JSON for "Hi" (mirrored)
2 python word_to_path.py
3
4 # Or import and call with any word:
5 from word_to_path import create_mirrored_word_json
6
7 json_output = create_mirrored_word_json("HELLO", mirror=True)
8 print(json_output)
9
10 # Then paste the JSON into the PATH tab -> Import from JSON
11 # The IK solver automatically finds joint angles for each Cartesian
   point
```

Listing D.1: Using word_to_path.py to generate a handwriting path

Bibliography

- [1] J. Denavit and R. S. Hartenberg, “A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices,” *Journal of Applied Mechanics*, vol. 22, pp. 215–221, Jun. 1955.
- [2] Y. Nakamura and H. Hanafusa, “Inverse Kinematic Solutions with Singularity Robustness for Robot Manipulator Control,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.
- [3] C. W. Wampler, “Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least Squares Methods,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, Jan./Feb. 1986.
- [4] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London: Springer, 2009.
- [5] Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.0, Espressif Systems, Shanghai, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [6] NXP Semiconductors, *PCA9685: 16-Channel, 12-Bit PWM Fm+ I2C-Bus LED Controller*, Product Data Sheet Rev. 4, NXP Semiconductors, Eindhoven, 2015.
- [7] Three.js Authors, *Three.js — JavaScript 3D Library*, r128, 2021. [Online]. Available: <https://threejs.org>
- [8] Elec Freaks, *Ultrasonic Ranging Module HC-SR04*, Datasheet Rev. 1.0, 2010.